

Campus Connect — A Closed-Community Social & Academic Networking Platform

Project Proposal for UCS503
Submitted to: Dr. Jeelani Asif
Thapar Institute of Engineering and Technology

Prakhar Saxena, CSED* Aayushmaan Singh Meyan, CSED[†]
Divyansh Jasrotia, CSED[‡]

August 15, 2026

1 Higher-Order Goal

... secondary framing

This project aims to provide colleges, schools, and organisations with a private, verified, and structured digital environment that consolidates social networking and academic management into a single platform. The broader goal is to eliminate the fragmentation caused by using general-purpose public platforms for institutional communication, and to replace it with a purposeful, role-aware, and AI-safeguarded community space that serves students, teachers, and administrators alike.

2 Time-to-Value

... primary heuristic

- Meaningful increments will be delivered by the end of each weekly sprint, beginning with core authentication and profile features and advancing to real-time communication and AI moderation.
- Fast feedback is prioritised over perfection: each completed module will be reviewed (and, where possible, CI-backed) before the next phase begins.
- Success in the first iteration is defined by a working login, profile, and social feed — enough for a real user to sign in, post, and connect.

3 Problem Statement

Educational institutions and closed organisations currently face the following challenges when communicating and collaborating digitally:

- **No dedicated platform:** Colleges and schools rely on scattered public apps — WhatsApp groups, Instagram, and email chains — that are not designed for institutional or academic use.

*Roll No: 1024240019, Email: psaxena.be24@thapar.edu

[†]Roll No: 1024240142, Email: ameyan.be24@thapar.edu

[‡]Roll No: 1024240008, Email: djasrotia.be24@thapar.edu

- **Information gets lost:** Course materials, exam schedules, and placement updates are drowned in irrelevant personal content on general social media.
- **No internal trusted network:** Students and faculty have no way to build a verified, private network within their own community.
- **No access control:** Public platforms cannot restrict membership to verified community members only.
- **No content safety layer:** Institutions have no mechanism to detect or remove inappropriate, vulgar, or policy-violating content posted by members.
- **Fragmented communication:** Academic tools (timetables, assignments, resources) and social communication exist in completely separate, unconnected applications.

Why this matters now (contextual relevance): These inefficiencies lead to missed deadlines, lost resources, poor academic coordination, and an unsafe communication environment — all of which directly impact student performance and institutional effectiveness.

4 Proposed Solution

... Software Proposition

4.1 Overview

Build a **web-based** closed-community social and academic networking platform with the following components:

- **Private and verified access:** only approved, verified members can join a community instance; admins control the join policy.
- **Social communication:** a structured social feed, friend system, direct messaging, and real-time notifications.
- **Academic utility:** a course materials hub, shared schedule board, and evaluation tracker for teachers and students.
- **Admin safety and control:** a full admin panel with user management, access control, content moderation, and an AI agentic moderation layer.

4.2 Core Workflow

1. Admin creates a community instance, sets the join policy, and adds the first users.
2. Users sign up via institutional email or invite link, complete email verification, and set up their profile.
3. Students join batch, lab, or subject groups; access course materials and the schedule board.
4. Teachers publish lectures, exams, and evaluations on the group calendar and upload course resources.
5. Members post in the social feed, connect as friends, and communicate via direct messages.
6. Every post or comment is scanned by the AI moderation agent; flagged content is hidden and queued for admin review.

4.3 Operational Constraints for an Educational Setting

- Fully browser-based — no app installation required; responsive design usable on desktops, tablets, and smartphones.
- Avoid dependence on advanced research algorithms; focus on workflow, automation, and system correctness.
- Designed for deployment on standard managed hosting and cloud database services.

5 Features and Functions

... module breakdown

The platform is structured into eleven functional modules, each solving a specific institutional problem. This is an engineering course project, so the emphasis below is on scalable administrative and workflow design rather than novel algorithm research.

5.1 Authentication and Access Control

Users register using their institutional email or an admin-generated invite link. Email verification confirms identity. The system supports role assignment (Admin, Teacher, Student), domain whitelisting, invite-link expiry control, and JWT-based secure session management with password reset.

5.2 Profile Management

Every member receives a personal profile page displaying their name, avatar, bio, department, batch, and role badge. Users can edit all profile fields at any time. The profile also shows an activity summary of posts made, groups joined, and materials uploaded.

5.3 Social Feed

Members create posts with text and image attachments. Posts can be scoped to the full community or to a specific group. Interactions include likes and threaded comments. The feed aggregates content from friends and joined groups in reverse-chronological order. Authors and admins can delete any post.

5.4 Friend System

Users send, accept, or reject friend requests. They can remove connections, view a full friends list, see pending requests, and receive suggestions based on shared batch, department, or mutual friends.

5.5 Academic Groups

Admins and teachers create groups for batches, labs, subjects, or clubs. Students browse and join open groups or accept invites. Each group has its own feed, a materials section, and a schedule board. Group admins can manage member roles and remove members as needed.

5.6 Course Materials Hub

Teachers and students upload PDFs, notes, images, and external links, organised into subject or topic folders within each group. Members can preview PDFs in-browser, download files, and search materials by name, subject, or uploader. An upload history log tracks all contributions.

5.7 Schedule Board

A shared calendar is available at the group level. Teachers add lectures, exams, and evaluation deadlines. Students add personal tasks and reminders visible only to themselves. Events are colour-coded by type. A notification is triggered whenever a new event is added to a group calendar.

5.8 Direct Messaging

Members exchange real-time private messages. Messages are delivered instantly via WebSockets, support image attachments, and display read receipts (delivered and seen states). Users have an inbox with unread badge counts and in-conversation search.

5.9 Notifications

A real-time notification bell covers all platform activity: friend requests, post likes and comments, new messages, group invitations, and schedule events. Notifications can be marked as read individually or all at once, and each links directly to the relevant content.

5.10 Admin Module

User Management: The admin views all members in a table with role, status, and join date. Actions include verifying new accounts, suspending or permanently removing members, changing roles, and bulk operations on multiple users.

Access Control: The admin sets the platform join policy as Open, Domain-Restricted, or Invite-Only. Invite links are generated with configurable expiry dates and usage limits and can be revoked at any time. Email domain whitelisting enforces institutional boundaries.

Content Moderation: The admin reviews AI-flagged content in a dedicated queue, manually flags or removes any post or account, and views a full audit log of all moderation actions. Members can submit post reports that appear in the admin dashboard.

5.11 AI Agentic Content Moderation

Every post and comment is automatically scanned by an AI agent immediately upon submission. The agent detects vulgarity, hate speech, harassment, and policy violations. Flagged content is auto-hidden from the feed and the admin is notified instantly with a written justification for the flag. Admins approve or permanently remove flagged content. A sensitivity control (Strict, Moderate, Lenient) and a global on/off toggle are available in the admin settings; the AI layer never auto-deletes an account, it only flags for human review.

6 Solution Approach

... Engineering focus

6.1 Tech Stack

Component	Technology
Frontend Framework	Next.js 14 (React, App Router)
Language	TypeScript
Styling	Tailwind CSS
Backend / API	Next.js API Routes (Node.js)
Database	PostgreSQL
ORM	Prisma
Authentication	NextAuth.js

Component	Technology
Real-Time Engine	Socket.io
File Storage	Cloudinary
AI Moderation	OpenAI GPT-4o / Google Gemini API
Hosting	Vercel (App) + Railway (Database)

The system follows a standard 3-tier architecture: a reactive frontend, a REST API backend, and a managed relational database. All real-time events are handled via a Socket.io server co-hosted with the backend.

6.2 CI/CD and Fast Delivery Enabler

CI/CD will be used to:

- Automatically build, lint, and run unit/integration tests on every change (GitHub Actions).
- Run Prisma migrations against a staging database as part of the pipeline, so schema drift is caught before release.
- Produce repeatable deployment artifacts and auto-deploy the frontend/API to a Vercel preview environment and the database to Railway staging.
- Enable safe, frequent releases of incremental features in line with the weekly sprint cadence.

7 Evaluation Criteria

... measurable and attributable

We define success using measurable metrics tied to the core workflow.

7.1 Primary Metric

Feature completeness: all 8 deliverables are functional and accessible to their respective user roles — Admin, Teacher, and Student — within the 6-week timeline.

- **Target:** 8/8 deliverables demoable by end of Week 6.
- **Attribution:** verified against the deliverable checklist and role-based acceptance tests at each sprint review.

7.2 Secondary Metrics

- **Real-time latency:** direct messages and notifications are delivered within 1 second under standard network conditions.
- **AI moderation accuracy:** the AI agent correctly flags clearly policy-violating content without excessive false positives, verified through a test batch of sample posts.
- **Role isolation:** students cannot access admin- or teacher-exclusive functions; verified through access-control testing.
- **Responsiveness:** the platform is fully usable on a standard mobile browser with no broken layouts.

7.3 Pilot Validation Plan

... *high-level*

- Recruit 10–20 pilot users (a mix of students and teachers) from a single class, lab section, or club to seed one community instance.
- Run a short User Acceptance Testing (UAT) pass covering sign-up, posting, group joining, messaging, and (for teacher/admin roles) moderation and scheduling.
- Collect baseline pain points via a short pre-pilot survey and compare against post-pilot feedback (1–5 clarity/usability rating) within the iteration window.

8 Scalability

... *with foresight*

1. **Scalable (theoretically):** The platform is built with a stateless API design, pagination on all list queries, and indexed database fields for common operations. This allows it to support a growing number of users, posts, and messages without architectural changes.
2. **Multi-instance deployable:** The platform is designed to be deployed as a separate instance for each institution, with no shared data between communities. Adding a new college or organisation requires only a new deployment.
3. **Operationally deployable (practically):** Using Vercel for the app and Railway for the managed database ensures repeatable, scalable deployments without custom server infrastructure, backed by the CI/CD pipeline for staging/production.

9 Engine Availability Heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Full-stack web framework for reactive pages and REST API routes (Next.js).
- Managed relational database with a type-safe ORM (PostgreSQL + Prisma).
- Authentication-as-a-library approach (NextAuth.js) instead of hand-rolled session handling.
- Real-time engine for messaging/notifications (Socket.io) instead of a custom pub/sub layer.
- Managed object storage for uploads and attachments (Cloudinary).
- Hosted LLM APIs for content moderation (OpenAI / Gemini) instead of training a custom classifier.
- CI runner and preview/staging deployment targets (GitHub Actions, Vercel, Railway).

We will use these mature, off-the-shelf components to focus engineering effort on workflow design, role correctness, and measurement, rather than on inventing new infrastructure.

10 Project Scope and Deliverables

... *iteration-friendly*

10.1 Initial Deliverable

... *first iteration*

- **Deliverable 1 — Authentication and User System:** sign-up, login, and email verification with role-based access for Admin, Teacher, and Student; invite-link onboarding, domain restriction, and secure session management.

- **Deliverable 2 — User Profiles and Social Network:** full personal profile pages with bio, avatar, and academic metadata; a working friend system with request flows and suggestions; a social feed with post creation, image upload, likes, and threaded comments.
- CI: build + lint + backend unit tests on every push.
- CD to a Vercel preview environment with a single pipeline job.

10.2 Subsequent Deliverables

- **Deliverable 3 — Academic Groups and Course Materials:** group creation/management for batches, labs, subjects, and clubs with role-based member management; a course materials hub with file upload, subject-folder organisation, in-browser PDF preview, download, and search.
- **Deliverable 4 — Schedule Board and Task Management:** an interactive shared calendar for group-level academic events; teacher tools for lectures, exams, and evaluations; student personal task board; colour-coded event types and notification triggers.
- **Deliverable 5 — Real-Time Communication:** one-on-one direct messaging with real-time delivery, image sharing, read receipts, and inbox management; a live notification system covering all platform activity types.
- **Deliverable 6 — Admin Control Panel:** user management, access-control configuration, content moderation, manual flagging, moderation history, and member-submitted reports.
- **Deliverable 7 — AI Moderation Agent:** automatic flagging/hiding of policy-violating posts and comments, admin notification with AI-generated reasoning, and sensitivity configuration.
- **Deliverable 8 — Deployed Web Application:** fully responsive, browser-accessible application, hosted on a production server with a live database, file storage, and complete technical documentation and user guide.

10.3 Week-by-Week Timeline

Week	Tasks	Deliverable
Week 1	Project scaffold, database schema, authentication system, role-based routing	Deliverable 1: Auth & User System
Week 2	Profile pages, friend system, social feed, post creation and engagement	Deliverable 2: Profiles & Social Network
Week 3	Academic groups, group management, course materials hub	Deliverable 3: Groups & Materials
Week 4	Schedule board, calendar, teacher and student task management	Deliverable 4: Schedule Board
Week 4–5	Real-time DM via Socket.io, notification system	Deliverable 5: Real-Time Communication

Week	Tasks	Deliverable
Week 5	Admin dashboard, user management, access control, moderation panel	Deliverable 6: Admin Control Panel
Week 6	AI moderation agent integration, pilot/UAT, bug fixes, deployment	Deliverables 7 & 8: AI Agent + Deployment

Total Estimated Duration: 6 Weeks (includes a buffer within Week 6 for pilot feedback and fixes).

11 Risks and Mitigations

- **AI API cost:** OpenAI or Gemini API calls are made per post. Mitigation: implement rate limiting and a per-community daily call budget; fall back to a rule-based filter if the budget is exceeded.
- **Real-time complexity:** Socket.io server management can introduce bugs in message delivery. Mitigation: build and test the DM system in isolation before integrating with notifications.
- **File storage size:** Cloudinary’s free tier has bandwidth limits. Mitigation: enforce file size limits (max 10 MB per upload) and file-type restrictions.
- **Role enforcement gaps:** misconfigured API routes may expose admin or teacher actions to students. Mitigation: role-guard middleware applied at the API layer, tested with dedicated access-control test cases.
- **Timeline slippage:** six features in six weeks is ambitious. Mitigation: Deliverables 7 and 8 are scheduled last, and AI moderation can be toggled off and replaced with a manual-only moderation flow if time runs short.
- **Data privacy / consent:** minimise collected data during the pilot; keep attachments optional; store only what is needed for the enrolled community.

12 Summary

Campus Connect proposes a deployable, web-based social and academic networking platform built exclusively for closed communities such as colleges and schools. It solves six real institutional problems — fragmented communication, lost academic information, lack of internal networking, no access control, no content safety, and tool fragmentation — through eleven integrated modules and eight concrete deliverables. The engineering approach prioritises iterative delivery, with a functional and demonstrable increment available at the end of each week, backed by CI/CD and a mature, off-the-shelf tooling stack rather than custom infrastructure. By combining a social feed, academic tools, real-time communication, an admin control panel, and an AI-powered moderation agent, Campus Connect delivers a complete, institutional-grade platform that no general-purpose social network currently provides.